Name:

PRN:

Experiment no 9

Title: CUDA Fundamentals — GPU Architecture, Threads, Blocks, Grids and Basic Kernels

1. Aim

To understand the fundamentals of CUDA programming and GPU-based parallel execution by studying the CUDA execution model and implementing basic CUDA kernels using threads, blocks, and grids.

2. Objectives

After completing this assignment, students will be able to:

- Understand the basic architecture of CPU and GPU.
- Explain the CUDA programming model.
- Identify the roles of host and device.
- Understand threads, blocks, grids and thread hierarchy.
- Write and execute basic CUDA kernels.
- Use CUDA memory management functions.
- Use thread and block indices to distribute work.
- Implement basic data-parallel operations using CUDA.
- Compare sequential, OpenMP and CUDA execution.
- Measure basic GPU performance.
-

3. Why Are We Learning This?

This massive parallelism makes GPUs useful for:

- Artificial intelligence and deep learning
- Computer vision
- Image and video processing
- Scientific computing
- Molecular simulation
- Numerical computation
- Data analytics
- HPC simulations

CUDA provides a programming model through which programmers can use NVIDIA GPUs for general-purpose computation.

4. CUDA Programming Model

4.1 Host and Device

In CUDA programming:

- **Host** generally refers to the CPU and its memory.
- **Device** generally refers to the GPU and its memory.

The host program controls the overall application and launches kernels on the GPU.

4.2 CUDA Kernel

A CUDA kernel is a function executed by many GPU threads.

A kernel is declared using:

```
__global__ void kernel_name(...)
{
    // GPU code
}
```

It is launched from the host using:

```
kernel_name<<<number_of_blocks, threads_per_block>>>(...);
```

4.3 Threads

A CUDA thread represents one execution instance of a kernel.

Each thread can identify itself using:

```
threadIdx.x
```

For example:

```
int tid = threadIdx.x;
```

5.4 Blocks

Threads are organized into blocks.

For example:

```
Block 0
 ├── Thread 0
 ├── Thread 1
 ├── Thread 2
 └── Thread 3

Block 1
 ├── Thread 0
 ├── Thread 1
 ├── Thread 2
 └── Thread 3
```

The block ID can be obtained using:

```
blockIdx.x
```

The number of threads in a block can be obtained using:

```
blockDim.x
```

4.5 Global Thread ID

For a one-dimensional grid, a common global thread ID is:

tid=blockIdx.x × blockDim.x + threadIdx.xtid

In CUDA:

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
```

This calculation is fundamental for assigning input data elements to GPU threads.

5. Software and Hardware Requirements

Hardware

- NVIDIA GPU supporting CUDA
- CUDA-capable system or HPC cluster

Software

- NVIDIA CUDA Toolkit
- CUDA compiler (`nvcc`)
- C/C++ compiler
- Linux/Windows environment as provided by the laboratory

Verify the installation using appropriate CUDA/NVIDIA system commands.

Record:

- GPU model
- GPU memory
- Compute capability, if available
- CUDA version
- Compiler version
- CPU information
- Operating system

Section A – Basics: CUDA Environment and Execution Model

Task 1: Explore the GPU Environment

Write a simple CUDA/C++ program or use the available CUDA utilities to identify the GPU available on your system.

Record:

Parameter	Value
GPU Name	
GPU Memory	
CUDA Version	
Compute Capability	
Number of SMs, if available	
Maximum Threads per Block	
Maximum Grid Dimensions	

Questions

1. What GPU is available on your system?
2. What is the maximum number of threads supported in one block?
3. What is an SM?
4. Why are GPU resources different from CPU cores?

Task 2: First CUDA Program — Hello from GPU

Write a CUDA program that launches a kernel in which every thread prints:

```
Hello from GPU Thread <thread_id>
```

Use:

- 1 block
- 8 threads

Then modify the program to use:

- 1 block, 16 threads
- 2 blocks, 8 threads per block
- 4 blocks, 8 threads per block

Observe

Record:

Blocks	Threads/Block	Total Threads	Output Observation
1	8	8	
1	16	16	
2	8	16	
4	8	32	

Think

Does changing the number of blocks change the thread ID values? Explain.

Task 3: Understand Thread and Block IDs

Write a CUDA program that displays:

```
Block ID
Thread ID within Block
Global Thread ID
```

Use:

```
blockIdx.x
threadIdx.x
blockDim.x
```

Calculate:

GlobalID=blockIdx.x × blockDim.x + threadIdx.x

Run with:

```
2 blocks × 4 threads
```

Expected conceptual structure:

```
Block 0:
Thread 0 → Global ID 0
Thread 1 → Global ID 1
Thread 2 → Global ID 2
Thread 3 → Global ID 3

Block 1:
Thread 0 → Global ID 4
Thread 1 → Global ID 5
Thread 2 → Global ID 6
Thread 3 → Global ID 7
```

Section B – Moderate Application: Data Parallelism

Task 4: Vector Addition Using CUDA

Step 1 – Sequential Version

Implement vector addition on the CPU.

Measure execution time.

Step 2 – CUDA Version

Implement vector addition using a CUDA kernel.

Each thread should calculate one element:

```
Thread 0 → C[0]
Thread 1 → C[1]
Thread 2 → C[2]
...
```

Use:

```
int id = blockIdx.x * blockDim.x + threadIdx.x;
```

Ensure that:

```
if (id < N)
```

is used to avoid accessing elements outside the array.

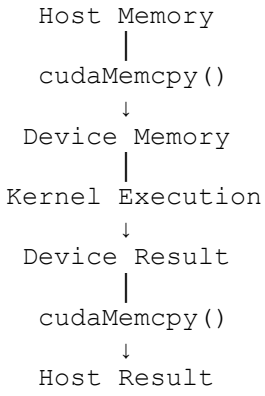
Task 5: CUDA Memory Management

Implement vector addition using CUDA device memory.

Study and use:

```
cudaMalloc()
cudaMemcpy()
cudaFree()
```

Understand the data movement:



Record

Identify which operations occur on:

- Host
- Device

and explain the purpose of each CUDA memory-management function.

Task 6: Vector Addition with Different Configurations

Run the vector addition program with different block sizes.

Use:

```
64 threads/block
128 threads/block
256 threads/block
512 threads/block
```

For a fixed vector size, calculate the number of blocks required:

Vector Size	Threads/Block	Blocks	Execution Time
1,000,000	64		
1,000,000	128		
1,000,000	256		
1,000,000	512		

Analyze

- Does increasing threads per block always improve performance?
- What happens when the number of threads is not an exact multiple of NN?
- Why is the boundary check necessary?

Section C – Challenge: Understanding GPU Parallelism

Task 7: Vector Addition — Sequential vs OpenMP vs CUDA

Implement vector addition using three approaches:

Version 1

Sequential C/C++

Version 2

OpenMP

Version 3

CUDA

Use the same input size and verify that all three produce the same result.

Suggested vector sizes:

- 10510^5
- 10610^6
- 10710^7
- 10810^8, subject to available system resources

Measure execution time.

Prepare:

Vector Size	Sequential	OpenMP	CUDA
10510^5			
10610^6			
10710^7			
10810^8			

Analyze

1. Which implementation is fastest for smaller input sizes?
2. What happens as the input size increases?

3. Why might CUDA have overhead for small problems?
4. What additional overhead is introduced by host-device data transfer?
5. Why can CUDA become advantageous for highly parallel workloads?

Task 8: One-Dimensional Array Processing

Choose **one** of the following applications and implement it using CUDA:

Option A – Vector Scaling
Option B – Element-wise Vector Multiplication
Option C – Array Squaring

- CUDA kernel
- Global thread ID
- Device memory
- Host-device data transfer

Compare the CUDA implementation with its sequential version.

7. Performance Analysis

For the CUDA programs, measure:

Execution Time

Measure the time required for kernel execution.

Where appropriate, separately discuss:

1. Host-to-device transfer
2. Kernel execution
3. Device-to-host transfer

This distinction is important because total application time is not necessarily the same as GPU kernel execution time.

Calculate Speedup for all programs.

8. Required Performance Graphs

Prepare:

Graph 1

Input Size vs Execution Time

Compare:

- Sequential
- OpenMP
- CUDA

Graph 2

CUDA Block Size vs Execution Time

Graph 3

Input Size vs CUDA Speedup

9. Correctness Verification

For every CUDA program:

1. Generate an input.
2. Execute the sequential version.
3. Execute the CUDA version.
4. Compare the results.

The output should be verified programmatically where possible.

For example:

```
CPU Result = ...
GPU Result = ...
Difference = ...
```

State whether the results are identical or within an acceptable numerical tolerance.

10. AI/HPC Case Study

Choose one application where GPU acceleration is widely useful.

Suggested areas:

- Deep learning
- Image processing
- Computer vision
- Scientific computing
- Molecular dynamics
- Medical imaging
- Financial computation
- Large-scale data analytics
- Signal processing

Write **300–500 words** covering:

1. The computational problem.
2. Why the problem contains substantial parallelism.
3. How GPU threads can be used.
4. What data can be processed independently.
5. Why GPU acceleration can be useful.
6. What limitations or bottlenecks may exist.
7. The role of CUDA or GPU computing in the application.

11. Questions

1. What is CUDA?
2. What is the difference between host and device?
3. What is a CUDA kernel?
4. What is a thread?
5. What is a block?
6. What is a grid?
7. How is the global thread ID calculated?
8. Why are threads organized into blocks?
9. What is the purpose of `cudaMalloc()`?
10. What is the purpose of `cudaMemcpy()`?
11. Why is `cudaFree()` required?
12. Why is a boundary check such as `id < N` necessary?
13. What happens when the number of data elements is not divisible by the number of threads?

14. Why can GPU execution be slower than CPU execution for small workloads?
15. What is host-device data transfer overhead?
16. Why can GPUs be effective for AI and HPC workloads?
17. Does increasing the number of GPU threads always increase performance? Explain.
18. What is the difference between a CPU core and a CUDA thread?
19. How is CUDA different from OpenMP?
20. Why is understanding the GPU execution hierarchy important for performance optimization?

12. Reflection

Answer the following based on your observations:

1. What was your main learning from this assignment?
2. How is CUDA programming different from OpenMP programming?
3. What role does the thread ID play in data-parallel computation?
4. What did you observe when changing the block size?
5. Why does CUDA require explicit memory management?
6. When would you prefer GPU execution over CPU execution?
7. What performance bottlenecks did you observe?
8. What CUDA concepts would you need to learn next to improve performance?

13. Deliverables

A. Programs

Submit:

1. GPU information program/output
2. CUDA Hello World
3. Thread and block ID program
4. CUDA vector addition
5. CUDA memory management implementation
6. Vector addition with different block configurations
7. Sequential vs OpenMP vs CUDA comparison
8. One selected array-processing application

B. Report

Include:

- Aim
- Objectives
- System configuration
- CUDA environment details
- Theory
- Algorithms
- Source code
- Sample input/output
- Correctness verification
- Performance tables
- Graphs
- Speedup calculations
- Analysis
- AI/HPC case study
- Discussion-question answers
- Reflection